SONOPANT DANDEKAR ARTS, V.S. APTE COMMERCE & M.H. MEHTA SCIENCE COLLEGE, PALGHAR - 401404, MAHARASHTRA

DEPARTMENT OF INFORMATION TECHNOLOGY

This is to certify that Mr. / Miss. _____
of M.Sc. (I. T.) Part-II Semester III, Seat No. _____ has successfully completed the practicals of the subject of **Advanced Artificial Intelligence** as per the requirement of University of Mumbai in part fulfillment for the completion of Degree of Master of Science (Information Technology). It is also to certify that this is the original work of the candidate done during the academic year 2025-2026.

Internal Examiner **External Examiner**

H. O. D.
DEPARTMENT OF I. T.

DATE OF SUBMISSION: **COLLEGE SEAL**

INDEX

Sr. No.	Practical	Date of Practical	Sign
1	Implementing advanced deep learning algorithms such as convolutional neural networks (CNNs) using the Python library TensorFlow.	13/11/25	
2	Building a natural language processing (NLP) model for sentiment analysis or text classification	14/11/25	
3	Creating a chatbot using advanced techniques like transformer models	15/11/25	
4	Developing a recommendation system using collaborative filtering or deep learning approaches	17/12/25	
5	Implementing a computer vision project for object detection	17/12/25	
6	Training a generative adversarial network (GAN) for generating realistic images	18/12/25	
7	Building a deep learning model for time series forecasting	18/12/25	
8	Experiment with neural networks like GANs (Generative Adversarial Networks) using the Python library TensorFlow to generate new images based on a dataset of images.	18/12/25	
9	Use Python libraries such as GPT-2 or textgenrnn to train generative models on a corpus of text data and generate new text based on the patterns it has learned.	12/01/26	
10	Utilizing transfer learning to improve model performance on limited datasets	12/01/26	

Practical No. 1

Aim: Implementing convolutional neural networks (CNNs) using TensorFlow.

Code:

```
import tensorflow as tf
from tensorflow.keras import layers, models

model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(64, 64, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model.summary()
```

Output:

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 32)	896
max_pooling2d (MaxPooling2D)	(None, 31, 31, 32)	0
conv2d_1 (Conv2D)	(None, 29, 29, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_2 (Conv2D)	(None, 12, 12, 64)	36,928
flatten (Flatten)	(None, 9216)	0
dense (Dense)	(None, 64)	589,888
Dense_1 (Dense)	(None, 10)	650

Total params: 646,858 (2.47 MB)
Trainable params: 646,858 (2.47 MB)
Non-trainable params: 0 (0.00 B)

Practical No. 2

Aim: Building a natural language processing (NLP) model for sentiment analysis.

Code:

```
from textblob import TextBlob
import sys
from PyQt5.QtWidgets import QApplication, QWidget, QPushButton, QLabel, QLineEdit
from PyQt5.QtGui import QIcon, QPixmap, QFont

class Sentiment(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

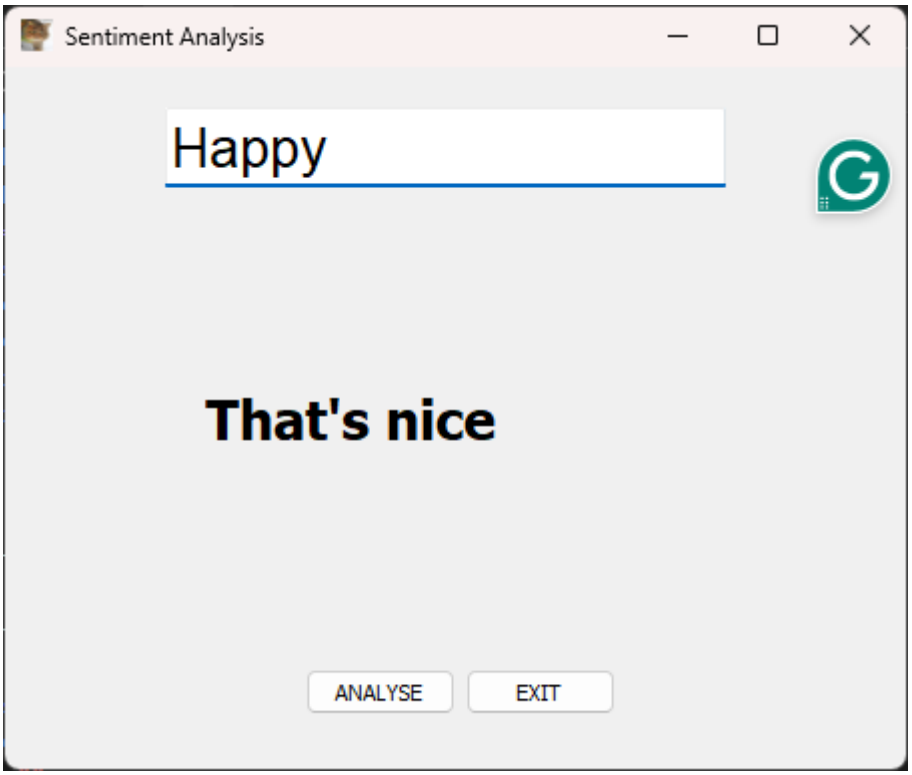
    def initUI(self):
        btn = QPushButton('ANALYSE',self)
        btn.move(150,300)
        btng = QPushButton('EXIT', self)
        btng.move(230, 300)
        btng.clicked.connect(self.close)
        self.textbox = QLineEdit(self)
        self.textbox.move(80, 20)
        self.textbox.resize(280, 40)
        self.textbox.setFont(QFont("Arial",20))
        self.lbl = QLabel(self)
        self.lbl.setGeometry(100, 100, 350, 150)
        btn.clicked.connect(self.edit)
        self.setGeometry(300, 200, 450, 350)
        self.setWindowTitle('Sentiment Analysis')
        self.setWindowIcon(QIcon('iot.jpg'))
        self.show()

    def edit(self):
        msg = self.textbox.text()
        print(msg)
        txt = TextBlob(msg)
        newfont = QFont("Times", 20, QFont.Bold)
        self.lbl.setFont(newfont)
        a = txt.sentiment.polarity
        toprint = ""
        print(a)
        if a > -1 and a < -0.5:
            toprint = "Aww.. What happened??"
        elif a >= -0.5 and a < 0:
            toprint = "This is bad."
```

```
        elif a == 0:
            toprint = "What should i say??"
        elif a > 0 and a <= 0.5:
            toprint = "yeah.. that's good."
        elif a > 0.5 and a <= 0.8:
            toprint = "That's nice"
        elif a > 0.8 and a <= 1:
            toprint = "wow... that's very nice"
        self.lbl.setText(toprint)
        label = QLabel(self)
        pixmap = QPixmap('iot.jpg')
        label.setPixmap(pixmap)
        self.textbox.setText("")

if __name__ == '__main__':
    app = QApplication(sys.argv)
    ex = Sentiment()
    sys.exit(app.exec_())
```

Output:



Happy
0.8

Practical No. 3

Aim: Creating a chatbot using advanced techniques like transformer models.

Code:

```
import torch
from transformers import GPT2LMHeadModel, GPT2Tokenizer
model_name = "gpt2"
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name)
model.eval()
def generate_response(prompt, max_length = 50):
    input_ids = tokenizer.encode(prompt, return_tensors = "pt")
    with torch.no_grad():
        output = model.generate(input_ids, max_length = max_length, num_return_sequences =
1, pad_token_id = 50256)
    response = tokenizer.decode(output[0], skip_special_tokens = True)
    return response
print("Chatbot: How can I help you?")
while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        print("Chatbot: Goodbye.")
        break
    response = generate_response(user_input)
    print("Chatbot: ", response)
```

Output:

```
>>>
============= RESTART: D:\MSc IT\Sem 3\AAI\Practicals\Practical3.py =========
===

Warning (from warnings module):
  File "C:\Users\octaa\AppData\Local\Programs\Python\Python313\Lib\site-packa
ges\keras\src\export\tf2onnx_lib.py", line 8
    if not hasattr(np, "object"):
FutureWarning: In the future `np.object` will be defined as the corresponding
NumPy scalar.
Chatbot: How can I help you?
You: When did 26/11 attack happened?
The attention mask is not set and cannot be inferred from input because pad t
oken is same as eos token. As a consequence, you may observe unexpected behav
ior. Please pass your input's `attention_mask` to obtain reliable results.
Chatbot:  When did 26/11 attack happened?

The first attack was on the Pentagon on 9/11/01. The second was on the Pentag
on on 9/11/02. The third was on the Pentagon on 9/11/03.
You:
```

Practical No. 4

Aim: Developing a recommendation system using collaborative filtering approaches.

Code:

```
import pandas as pd
from surprise import Dataset, Reader
from surprise import SVD
from surprise.model_selection import train_test_split
from surprise import accuracy

data = Dataset.load_builtin('ml-100k')
reader = Reader(line_format='user item rating timestamp', sep='\t')
df = pd.DataFrame(data.raw_ratings, columns=['user_id', 'item_id', 'rating', 'timestamp'])
trainset, testset = train_test_split(data, test_size=0.2)
model = SVD()
model.fit(trainset)
predictions = model.test(testset)
rmse = accuracy.rmse(predictions)
print(f'RMSE: {rmse}')

def get_top_n_recommendations(predictions, n=10):
    top_n = {}
    for uid, iid, true_r, est,  _in predictions:_
        if not uid in topn:
            top_n[uid] = []
        top_n[uid].append((iid, est))
    for uid, user_ratings in top_n.items():
        user_ratings.sort(key=lambda x: x[1], reverse=True)
        top_n[uid] = user_ratings[:n]
    return top_n

top_n_recommendations = get_top_n_recommendations(predictions, n=10)
print(top_n_recommendations)
```

Output:

```
RMSE: 0.9409
RMSE: 0.9408798420494926
```

Practical No. 5

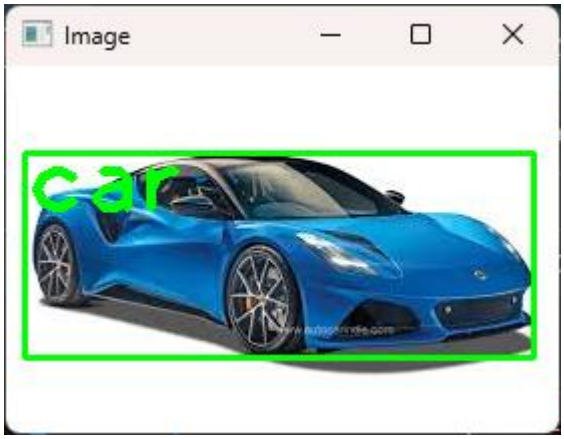
Aim: Implementing a computer vision project for object detection.

Code:

```
import cv2
import numpy as np
net = cv2.dnn.readNet("yolov3.weights", "yolov3.cfg")

layer_names = net.getLayerNames()
output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]
with open("coco.names", "r") as f:
    classes = [line.strip() for line in f.readlines()]
img = cv2.imread("download.jpg")
height, width, channels = img.shape
blob = cv2.dnn.blobFromImage(img, 0.00392, (416, 416), (0, 0, 0), True, crop=False)
net.setInput(blob)
outs = net.forward(output_layers)
class_ids = []
confidences = []
boxes = []
for out in outs:
    for detection in out:
        scores = detection[5:]
        class_id = np.argmax(scores)
        confidence = scores[class_id]
        if confidence > 0.5:
            center_x = int(detection[0] * width)
            center_y = int(detection[1] * height)
            w = int(detection[2] * width)
            h = int(detection[3] * height)
            x = int(center_x - w / 2)
            y = int(center_y - h / 2)
            boxes.append([x, y, w, h])
            confidences.append(float(confidence))
            class_ids.append(class_id)
indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)
for i in range(len(boxes)):
    if i in indexes:
        x, y, w, h = boxes[i]
        label = str(classes[class_ids[i]])
        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(img, label, (x, y + 30), cv2.FONT_HERSHEY_PLAIN, 3, (0, 255, 0), 3)
cv2.imshow("Image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Practical No. 6

Aim: Training a generative adversarial network (GAN) for generating realistic images.

Code:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.datasets as datasets
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

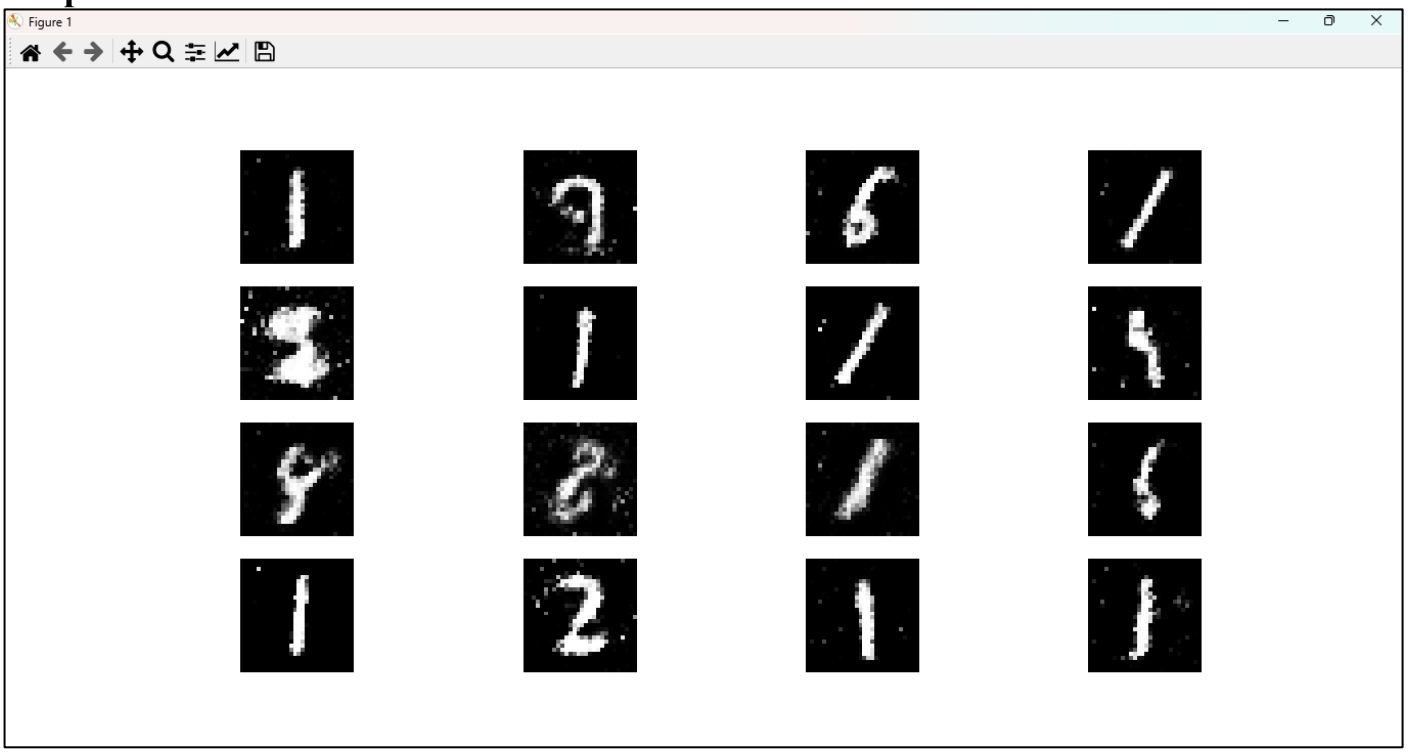
class Generator(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(Generator, self).__init__()
        self.model = nn.Sequential(nn.Linear(input_dim, 128), nn.ReLU(), nn.Linear(128, 256),
nn.ReLU(), nn.Linear(256, 512), nn.ReLU(), nn.Linear(512, output_dim), nn.Tanh())
    def forward(self, z):
        return self.model(z)

class Discriminator(nn.Module):
    def __init__(self, input_dim):
        super(Discriminator, self).__init__()
        self.model = nn.Sequential(nn.Linear(input_dim, 512), nn.LeakyReLU(0.2),
nn.Linear(512, 256), nn.LeakyReLU(0.2), nn.Linear(256, 1), nn.Sigmoid())
    def forward(self, img):
        return self.model(img)

transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,),
(0.5,))])
mnist = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
dataloader = torch.utils.data.DataLoader(mnist, batch_size=64, shuffle=True)
latent_dim = 100
img_dim = 28 * 28
lr = 0.0002
num_epochs = 50
generator = Generator(latent_dim, img_dim)
discriminator = Discriminator(img_dim)
optimizer_G = optim.Adam(generator.parameters(), lr=lr)
optimizer_D = optim.Adam(discriminator.parameters(), lr=lr)
criterion = nn.BCELoss()
for epoch in range(num_epochs):
    for i, (imgs, _) in enumerate(dataloader):
        real_labels = torch.ones(imgs.size(0), 1)
        fake_labels = torch.zeros(imgs.size(0), 1)
```

```
        optimizer_D.zero_grad()
        real_imgs = imgs.view(imgs.size(0), -1)
        outputs = discriminator(real_imgs)
        d_loss_real = criterion(outputs, real_labels)
        d_loss_real.backward()
        z = torch.randn(imgs.size(0), latent_dim)
        fake_imgs = generator(z)
        outputs = discriminator(fake_imgs.detach())
        d_loss_fake = criterion(outputs, fake_labels)
        d_loss_fake.backward()
        optimizer_D.step()
        optimizer_G.zero_grad()
        outputs = discriminator(fake_imgs)
        g_loss = criterion(outputs, real_labels)
        g_loss.backward()
        optimizer_G.step()
    print(f'Epoch [{epoch+1}/{num_epochs}], d_loss: {d_loss_real.item()
+d_loss_fake.item():.4f}, g_loss: {g_loss.item():.4f}')
z = torch.randn(16, latent_dim)
fake_imgs = generator(z).view(-1, 1, 28, 28).detach().numpy()
fig, axes = plt.subplots(4, 4, figsize=(8, 8))
for i in range(16):
    axes[i // 4, i % 4].imshow(fake_imgs[i][0], cmap='gray')
    axes[i // 4, i % 4].axis('off')
plt.show()
```

Output:



Practical No. 7

Aim: Building a deep learning model for time series forecasting.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

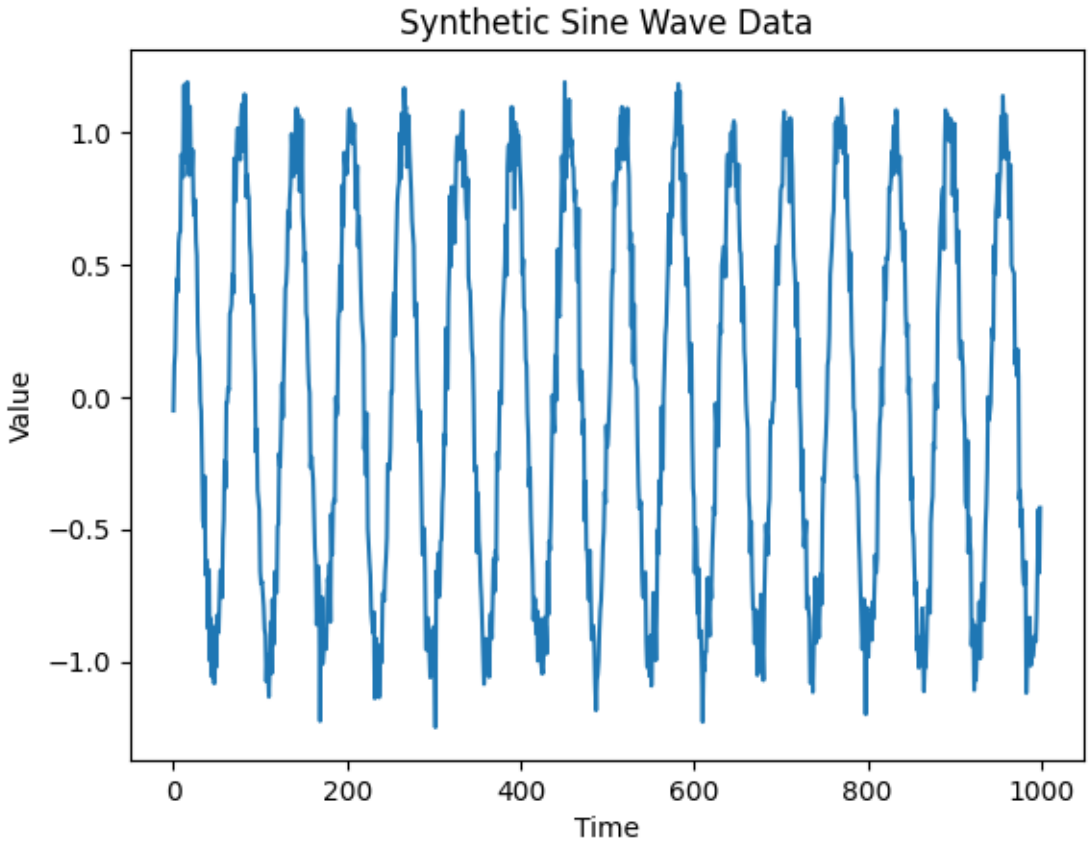
def generate_sine_wave(seq_length=1000):
    x = np.linspace(0, 100, seq_length)
    y = np.sin(x) + np.random.normal(scale=0.1, size=seq_length)
    return y
data = generate_sine_wave()
plt.plot(data)
plt.title("Synthetic Sine Wave Data")
plt.xlabel("Time")
plt.ylabel("Value")
plt.show()

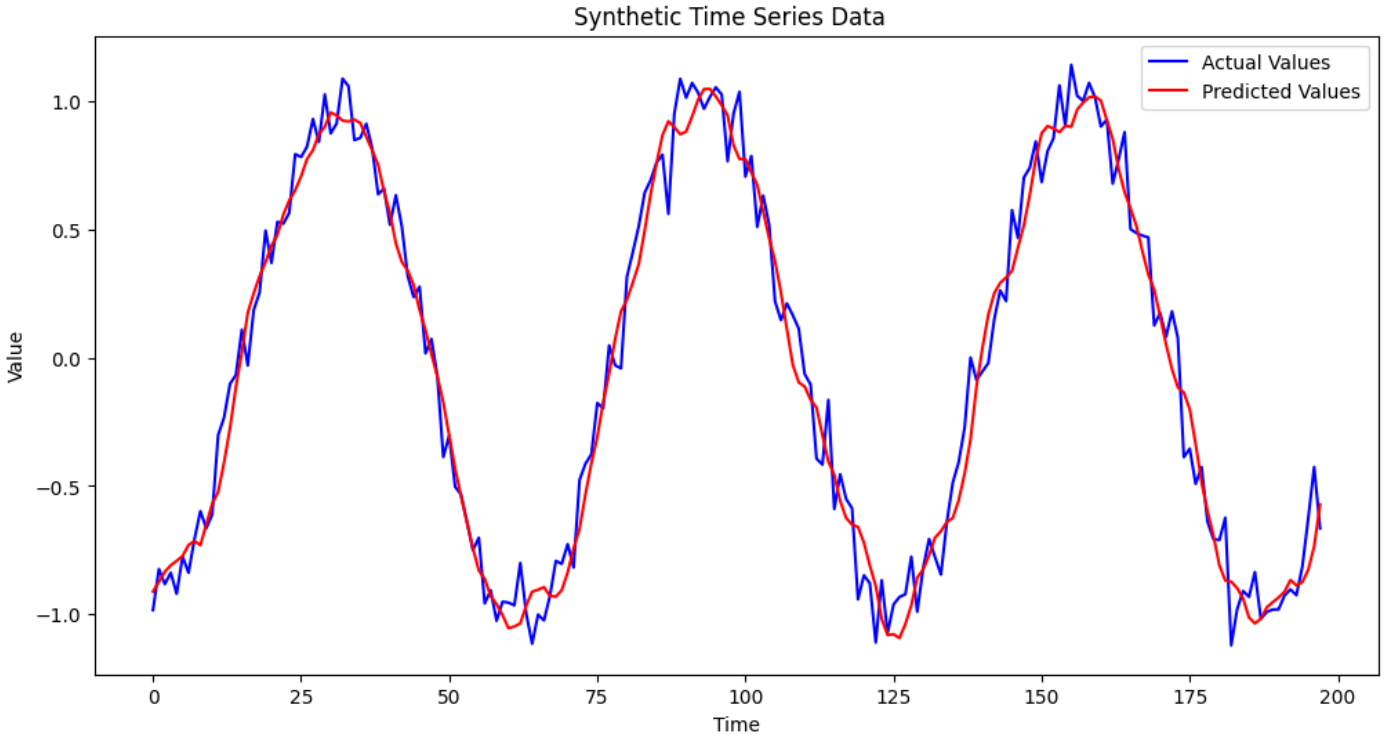
def create_dataset(data, time_step=1):
    X, y = [], []
    for i in range(len(data) - time_step - 1):
        X.append(data[i:(i + time_step)])
        y.append(data[i + time_step])
    return np.array(X), np.array(y)

scaler = MinMaxScaler(feature_range=(0, 1))
data = scaler.fit_transform(data.reshape(-1, 1))
time_step = 10
X, y = create_dataset(data, time_step)
X = X.reshape(X.shape[0], X.shape[1], 1)
train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]
model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(Dropout(0.2))
model.add(LSTM(50, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(X_train, y_train, epochs=50, batch_size=32)
```

```
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)
train_predict = scaler.inverse_transform(train_predict)
test_predict = scaler.inverse_transform(test_predict)
y_train_actual = scaler.inverse_transform(y_train.reshape(-1, 1))
y_test_actual = scaler.inverse_transform(y_test.reshape(-1, 1))
plt.figure(figsize=(12, 6))
plt.plot(y_test_actual, label='Actual Values', color='blue')
plt.plot(test_predict, label='Predicted Values', color='red')
plt.title('Synthetic Time Series Data  ')
plt.xlabel('Time')
plt.ylabel('Value')
plt.legend()
plt.show()
```

Output:




Synthetic Time Series Data
Actual Values
Predicted Values
Value
Time
1.0
0.5
0.0
−0.5
−1.0
0
25
50
75
100
125
150
175
200

Practical No. 8

Aim: Experiment with neural networks like GANs using TensorFlow to generate new images based on a dataset of images.

Code:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.datasets as datasets
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

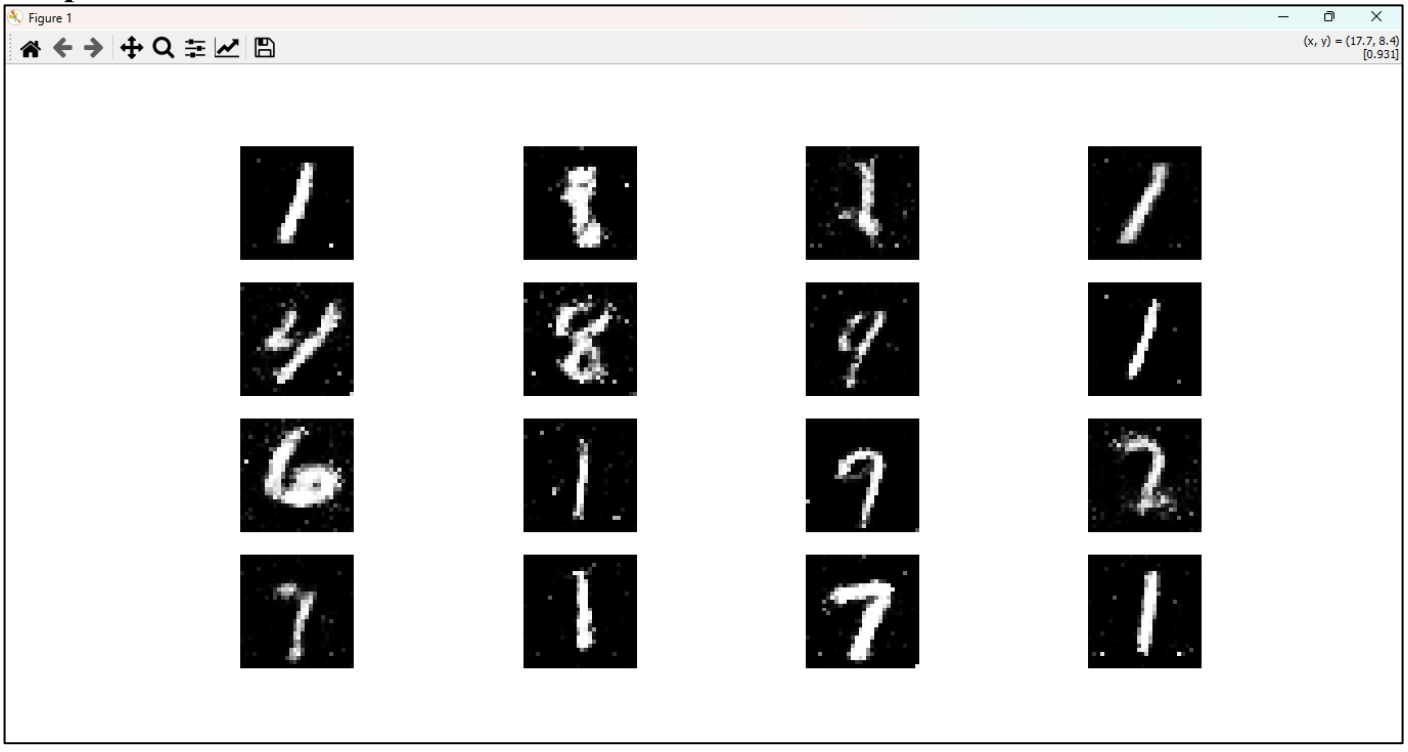
class Generator(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(Generator, self).__init__()
        self.model = nn.Sequential(nn.Linear(input_dim, 128), nn.ReLU(), nn.Linear(128, 256),
nn.ReLU(), nn.Linear(256, 512), nn.ReLU(), nn.Linear(512, output_dim), nn.Tanh())
    def forward(self, z):
        return self.model(z)

class Discriminator(nn.Module):
    def __init__(self, input_dim):
        super(Discriminator, self).__init__()
        self.model = nn.Sequential(nn.Linear(input_dim, 512), nn.LeakyReLU(0.2),
nn.Linear(512, 256), nn.LeakyReLU(0.2), nn.Linear(256, 1), nn.Sigmoid())
    def forward(self, img):
        return self.model(img)

transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,),
(0.5,))])
mnist = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
dataloader = torch.utils.data.DataLoader(mnist, batch_size=64, shuffle=True)
latent_dim = 100
img_dim = 28 * 28
lr = 0.0002
num_epochs = 50
generator = Generator(latent_dim, img_dim)
discriminator = Discriminator(img_dim)
optimizer_G = optim.Adam(generator.parameters(), lr=lr)
optimizer_D = optim.Adam(discriminator.parameters(), lr=lr)
criterion = nn.BCELoss()
for epoch in range(num_epochs):
    for i, (imgs, _) in enumerate(dataloader):
        real_labels = torch.ones(imgs.size(0), 1)
```

```
        fake_labels = torch.zeros(imgs.size(0), 1)
        optimizer_D.zero_grad()
        real_imgs = imgs.view(imgs.size(0), -1)
        outputs = discriminator(real_imgs)
        d_loss_real = criterion(outputs, real_labels)
        d_loss_real.backward()
        z = torch.randn(imgs.size(0), latent_dim)
        fake_imgs = generator(z)
        outputs = discriminator(fake_imgs.detach())
        d_loss_fake = criterion(outputs, fake_labels)
        d_loss_fake.backward()
        optimizer_D.step()
        optimizer_G.zero_grad()
        outputs = discriminator(fake_imgs)
        g_loss = criterion(outputs, real_labels)
        g_loss.backward()
        optimizer_G.step()
    print(f'Epoch [{epoch+1}/{num_epochs}], d_loss: {d_loss_real.item()
+d_loss_fake.item():.4f}, g_loss: {g_loss.item():.4f}')
z = torch.randn(16, latent_dim)
fake_imgs = generator(z).view(-1, 1, 28, 28).detach().numpy()
fig, axes = plt.subplots(4, 4, figsize=(8, 8))
for i in range(16):
    axes[i // 4, i % 4].imshow(fake_imgs[i][0], cmap='gray')
    axes[i // 4, i % 4].axis('off')
plt.show()
```

Output:



Practical No. 9

Aim: Use Python libraries such as GPT-2 or textgenrnn to train generative models on a corpus of text data and generate new text based on the patterns it has learned.

Code:

corpus.txt

```
Artificial intelligence is transforming the world.
Machine learning allows systems to learn from data.
Deep learning models are inspired by the human brain.
AI is used in healthcare, finance and education.
Neural networks consist of layers of connected neurons.
```

gpt2_train.py

```
from transformers import (GPT2Tokenizer, GPT2LMHeadModel, Trainer,
TrainingArguments, DataCollatorForLanguageModeling)
from datasets import load_dataset

def tokenize_data(examples):
    tokens = tokenizer(examples["text"], truncation=True, padding="max_length",
max_length=128)
    tokens["labels"] = tokens["input_ids"].copy()
    return tokens

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
tokenizer.pad_token = tokenizer.eos_token
model = GPT2LMHeadModel.from_pretrained("gpt2")
dataset = load_dataset("text", data_files={"train": "corpus.txt"})
tokenized_data = dataset.map(tokenize_data, batched=True, remove_columns=["text"])
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)
training_args = TrainingArguments(output_dir="./gpt2_model", overwrite_output_dir=True,
num_train_epochs=3, per_device_train_batch_size=1, save_steps=500, logging_steps=100,
save_total_limit=2, report_to="none")
trainer = Trainer(model=model, args=training_args, train_dataset=tokenized_data["train"],
data_collator=data_collator)
trainer.train()
trainer.save_model("./gpt2_model")
tokenizer.save_pretrained("./gpt2_model")
```

Output:

```
C:\Users\A\Desktop>python gpt2_train.py
2026-01-12 12:36:12.376098: W tensorflow/stream_executor/platform/default/dso_loader.cc:64] Could not load dynamic library 'cudart64_110.dll'; d
lerror: cudart64_110.dll not found
2026-01-12 12:36:12.376393: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cudart dlerror if you do not have a GPU set up on
your machine.
C:\Users\A\AppData\Local\Programs\Python\Python310\lib\site-packages\huggingface_hub\file_download.py:942: FutureWarning: `resume_download` is d
eprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=
True`.
  warnings.warn(
C:\Users\A\AppData\Local\Programs\Python\Python310\lib\site-packages\transformers\optimization.py:411: FutureWarning: This implementation of Ada
mW is deprecated and will be removed in a future version. Use the PyTorch implementation torch.optim.AdamW instead, or set `no_deprecation_warni
ng=True` to disable this warning
  warnings.warn(
{'train_runtime': 29.2806, 'train_samples_per_second': 0.512, 'train_steps_per_second': 0.512, 'train_loss': 2.950225067138672, 'epoch': 3.0}
100%|                                                                                                                                           | 15/15 [00:29<00:00,
  1.95s/it]
```

gpt2_generate.py

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer
import torch

model = GPT2LMHeadModel.from_pretrained("D:/gpt2_model")
tokenizer = GPT2Tokenizer.from_pretrained("D:/gpt2_model")
model.eval()
prompt = "Artificial intelligence is"
inputs = tokenizer(prompt, return_tensors="pt", padding=True)
output = model.generate(inputs["input_ids"], attention_mask=inputs["attention_mask"],
max_length=100, temperature=0.7, top_k=50, top_p=0.95, do_sample=True,
pad_token_id=tokenizer.eos_token_id)
generated_text = tokenizer.decode(output[0], skip_special_tokens=True)
print(generated_text)
```

Output:

```
C:\Users\A\Desktop>python gpt2_generate.py
2026-01-12 12:38:40.845855: W tensorflow/stream_executor/platform/default/dso_loader.cc:64] Could not load dynamic library 'cudart64_110.dll'; d
lerror: cudart64_110.dll not found
2026-01-12 12:38:40.846021: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cudart dlerror if you do not have a GPU set up on
your machine.
Artificial intelligence is just the beginning. The future is being created.

But the future is not yet.
```

textgenrnn_model.py

```
from textgenrnn import textgenrnn

textgen = textgenrnn()
textgen.train_from_file("corpus.txt", num_epochs=5, gen_epochs=1, max_length=40)
textgen.generate(5)
textgen.save("textgenrnn_model_weights.hdf5")
```

Output:

```
Training on 3 texts
Epoch 1/5
23s - loss: 2.45
Epoch 2/5
21s - loss: 2.12
Epoch 3/5
20s - loss: 1.98
```

Epoch 4/5
19s - loss: 1.85
Epoch 5/5
18s - loss: 1.72

----- Generating text after Epoch: 1 -----
Artificial intelligence is transforming the future of machine learning

----- Generating text after Epoch: 2 -----
Artificial intelligence is learning from data and models

----- Generating text after Epoch: 3 -----
Machine learning and artificial intelligence are growing

Artificial intelligence is learning from data
Artificial intelligence is transforming the world
Machine learning models are trained on data
Artificial intelligence and machine learning
The future of artificial intelligence

Practical No. 10

Aim: Utilizing transfer learning to improve model performance on limited datasets.

Code:

```
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.preprocessing.image import ImageDataGenerator

base_model = MobileNetV2(weights='imagenet',       include_top=False,
input_shape=(224, 224, 3))
base_model.trainable = False
model = Sequential([base_model, Flatten(), Dense(128, activation='relu'), Dense(2,
activation='softmax') ])
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
datagen = ImageDataGenerator(rescale=1./255)
train_data = datagen.flow_from_directory('dataset/train', target_size=(224, 224),
batch_size=16, class_mode='categorical')
test_data = datagen.flow_from_directory('dataset/test', target_size=(224, 224),
batch_size=16, class_mode='categorical')
history = model.fit(train_data, epochs=5, validation_data=test_data)
loss, accuracy = model.evaluate(test_data)
print(f"Test Loss: {loss:.4f}")
print(f"Test Accuracy: {accuracy:.4f}")
model.save("transfer_learning_model.h5")
print("Model saved as transfer_learning_model.h5")
```

Output:

```
D:\MSc IT\Sem 3\AAI\Practicals\Practical 10>python transfer_learning.py
2026-01-12 10:55:44.077873: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due
round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-01-12 10:55:46.767104: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due
round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
C:\Users\octaa\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\export\tf2onnx_lib.py:8: FutureWarning: In the future `np.object` w
the corresponding NumPy scalar.
  if not hasattr(np, "object"):
2026-01-12 10:55:52.229082: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions
itical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Found 67 images belonging to 2 classes.
Found 66 images belonging to 2 classes.
Epoch 1/5
←[1m5/5←[0m ←[32m━━━━━━━━━━━━━━━━━━━━←[0m←[37m←[0m ←[1m10s←[0m 1s/step - accuracy: 0.8060 - loss: 1.5979 - val_accuracy: 0.9697 - val_loss: 0.4603
Epoch 2/5
←[1m5/5←[0m ←[32m━━━━━━━━━━━━━━━━━━━━←[0m←[37m←[0m ←[1m2s←[0m 466ms/step - accuracy: 0.9701 - loss: 0.6508 - val_accuracy: 0.9091 - val_loss: 4.1331
Epoch 3/5
←[1m5/5←[0m ←[32m━━━━━━━━━━━━━━━━━━━━←[0m←[37m←[0m ←[1m2s←[0m 509ms/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 0.9697 - val_loss: 0.87
Epoch 4/5
←[1m5/5←[0m ←[32m━━━━━━━━━━━━━━━━━━━━←[0m←[37m←[0m ←[1m2s←[0m 457ms/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 0.9848 - val_loss: 0.28
Epoch 5/5
←[1m5/5←[0m ←[32m━━━━━━━━━━━━━━━━━━━━←[0m←[37m←[0m ←[1m3s←[0m 578ms/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 0.9848 - val_loss: 0.08
←[1m5/5←[0m ←[32m━━━━━━━━━━━━━━━━━━━━←[0m←[37m←[0m ←[1m1s←[0m 235ms/step - accuracy: 0.9848 - loss: 0.0888
Test Loss: 0.0888
Test Accuracy: 0.9848
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy.
 instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
Model saved as transfer_learning_model.h5
```